

Reflection

Reproducible NLP Pipeline in a Container — Slim Version

01 Which approach won, and when is a classical method still worth using?

Classical won by a mile: **0.967 / 0.920** (acc / macro F1) in 0.12 s, vs. the transformer's **0.450 / 0.426** in 137 s. Always predicting "ham" would have scored 0.866 — so the transformer was worse than nothing.

DistilBERT isn't a bad model, I just used it wrong. It's a sentiment classifier, and I mapped `POSITIVE → ham`, `NEGATIVE → spam`. That only works if spam sounds negative, which it doesn't — spam is usually cheerful ("You won!"), and short legit messages ("ok") read as negative.

A classical method is still worth it when the task is clear, the labels are clean, and a few thousand examples exist. TF-IDF + LogReg learns the obvious signal words ("free", "txt", "claim") in 100 ms. A transformer only earns its cost if the pretraining domain matches, or you fine-tune.

02 What's in your Docker image that wouldn't be in a venv?

- **The OS.** `python:3.11-slim` pins Debian bookworm and its glibc. PyTorch wheels are compiled against specific glibc versions — the same `torch==2.4.1` can `ImportError` on an older Linux.
- **Python itself.** `requirements.txt` doesn't pin the interpreter. My first run died on Python 3.13 because `torch==2.4.1` has no wheel for it. `FROM python:3.11-slim` fixes that.
- **The layout.** `python -m src.main` always finds the package because `COPY src/ ./src/` puts it in a known place.

The Python pin was the one that actually bit me, and it's invisible from `requirements.txt` alone.

03 One thing that broke, one thing that surprised you.

Broke: `train_test_split(train_size=5000, test_size=1000)` crashed because SMS Spam only has 5,572 messages, not 6,000. I copied the split sizes from the slides without checking the dataset. Dropped to 4,000 + 1,000.

Surprised: how badly the transformer lost. I expected ± 5 points; got -50 . The lesson: a leaderboard number tells you nothing about whether a model fits *your* data.